WASP Software Engineering, Module 2

Emanuele Miotto

September 2026

1 Introduction

My research is in cybersecurity for cyber-physical systems and embodied AI. This is a branch of security for machines that act in the physical world rather than only process data, such as robots, autonomous vehicles, and industrial equipment. I work across a few related sub-topics rather than one project. What connects them is a simple observation: embodied AI systems are built from many components working together, and that complexity means many points of attack. Each sub-topic looks at one of these vectors and asks whether the system can still notice that something has gone wrong and react. They are at very different stages, from finished to still only an idea, so I indicate below where each stands.

The first sub-topic, developed with my supervisor, is about resilience. We define what it means for a cyber-physical system to detect an attack, keep operating through it, and recover afterwards, and we evaluate this against a robot in simulation across a range of attack scenarios. The goal is to give resilience a concrete, measurable definition instead of something a system either has or lacks. This is the most mature part of my work, and the only one peer-reviewed and accepted at ACM CCS CPSIoTSec workshop. The camera ready will be available in a couple of days so I can't provide a reference as for now.

The second sub-topic is about speech. Many robots and industrial systems now use models like Whisper to interpret spoken commands. We are studying how these models can be poisoned during fine-tuning, so that ordinary background sounds, a car horn or a hydraulic lift, become hidden triggers that make the model output a phrase chosen by the attacker. We are also testing whether voice activity detection can catch these triggers before they cause harm. We have results, but the work is in progress and unpublished.

A related sub-topic targets a different component: the control policy. Robotic arms are more and more often controlled by policies trained through reinforcement learning (RL) instead of hand-written logic, and I am studying how those can be poisoned as well: through the reward signal, the actions taken during training, or the observed states, in ways that stay hidden until a specific trigger appears. This part is at an early stage: a taxonomy of the attack surface and first experiments, but no complete results yet.

2 Lecture Principles

2.1 Invariant and metamorphic testing

One idea that maps very closely onto my work is invariant and metamorphic testing from the quality assurance lecture. Instead of needing a ground-truth answer for every test case, the tester checks that the output of a system does not change under transformations that shouldn't matter. The example used in the lecture was a credit model that shouldn't change its decision when only the gender field is modified. This is the same property that our speech recognition attack breaks. A well-behaved model transcribing a spoken command shouldn't change its output because of unrelated background noise, such as a car horn or a hydraulic lift. That is the invariant a backdoor-poisoned model violates: the trigger sound flips the output to a phrase chosen by the attacker. Seen this way, the attack is an invariant violation, and

the defense we are testing, voice activity detection, is an attempt to enforce that invariant at runtime, by filtering out the parts of the input that shouldn't influence the transcription in the first place.

2.2 Hidden technical debt and training/serving skew

A second idea is the point from Sculley et al. from lecture 1 that data, and not only code, is a first-class engineering artifact, and that training/serving skew is a common source of silent failure. This describes our attack quite well from the side of the defender. The poisoning happens by manipulating the fine-tuning data, not the model architecture and not any line of code, so a code review would never catch it. The reason why environmental trigger sounds work at all is also a form of training/serving skew: the trigger behaves consistently in the curated fine-tuning data, but in deployment it appears inside a much messier acoustic environment, with industrial noise, other voices, and so on. That gap between how the model was built and how it is later used is what the paper describes as a quiet cause of degradation in ML systems.

3 Guest-lecture principles

3.1 AI opacity

The clearest connection here is the point from Frattini that AI systems generally cannot be formalized in the way traditional software can. They tend to be obscure and atomic instead of decomposable and transparent, and this is one of the main reasons why classic requirements engineering struggles with them. That opacity is the root cause of most of what I study. A fine-tuned Whisper model or a trained RL policy cannot be decomposed into inspectable parts the way hand-written code can, so a poisoned fine-tuning run or a poisoned reward signal produces a component that looks completely normal from the outside. There is nothing to review. My resilience-framework project is in a sense a direct answer to this problem: instead of trying to formalize the AI component itself, we formalize resilience at the system level. We define what it means to detect, tolerate, and recover from an attack, without having to open the opaque model in order to reason about whether the system as a whole still behaves safely.

3.2 Automated evaluation replacing review gates

From the lecture by Lenberg, the idea I keep coming back to is that during development, automated evaluation assisted by AI progressively replaces human review. This is a fairly accurate description of what a realistic defense against the attacks I study would have to look like. Nobody is going to listen manually to every audio stream that a robot receives in order to check for a suspicious background noise, and nobody is going to audit by hand every episode of an RL training run to look for a poisoned reward. The voice activity detection defense we are testing, and the resilience predicates from the other project, are both attempts to build that missing automated evaluation step. They are a review gate that operates at the speed at which these systems really run, because a human in the loop was never a realistic option here.

4 Data scientists, software engineers, and AI engineers

4.1 Data scientists vs software engineers

The chapters describe data scientists as coming from a statistics and machine-learning background, working in an exploratory, notebook-driven style, and evaluating their work mainly through accuracy on held-out data, with little attention to latency or operating cost. Software engineers are described as focused on delivering a working product under constraints, spending their time on requirements, architecture, testing, and deployment, and constantly trading off usability, maintainability, and cost. The book is open

about this being an oversimplification, but I think it holds up well as a description of tendencies rather than a hard rule.

In my own work I recognize this distinction in a very concrete way. Debugging a HuggingFace Datasets backend incompatibility so that a training pipeline runs reliably feels like pure software engineering. Designing the poisoning attack itself, choosing which trigger sounds to use, deciding how to shape the reward function and what counts as a successful attack, feels much closer to the data-science side, exploratory and driven by evaluation. I do not get to specialize in only one of these, which is closer to the "T-shaped" idea the reading mentions. Security research forces me to do a bit of both.

4.2 The system-wide view and my own research

This is where the reading connects most closely to what I do. The resilience-framework project is explicitly a system-level argument: resilience is a property of the whole cyber-physical system, whether it can detect, tolerate, and recover from an attack, and not of any single model or component. We test it against the full simulated robot instead of any isolated part.

The speech recognition work sits right at the boundary the book keeps returning to. The attack targets the ML component, but the mitigation we are testing, voice activity detection, is a non-ML component placed around the model in the pipeline. The same work also runs into the training/serving skew problem I described earlier, another case where reasoning only about the model would hide the real failure.

The RL and robotic-arm thread, still at an early stage, connects to the "lack of specification" problem. There is no ground-truth specification for what a correct reward function should look like, so a subtly poisoned reward can pass as a plausible design choice instead of an obvious bug. Similarly, the book argues that a model cannot really be called "correct", only good enough in an empirical sense. It also touches feedback loops: a slightly biased reward signal can reinforce itself during training, like the YouTube example in the book reinforces itself through user interactions.

4.3 Will the two roles converge?

My guess is that they will not fully merge as job titles, mostly because the observation about "unicorns" being rare reflects a real organizational incentive to specialize. Still, the overlap expected from each side will keep growing. Better MLOps tooling is already absorbing what required a dedicated software engineer, and AI coding assistants lower the bar for data scientists to write solid production code themselves. In security research it is difficult to pick a side, the skills required fall in the boundary.

4.4 AI Engineering

Based on the reading, AI Engineering is used most commonly for the automated-pipeline, deployment, and MLOps side, which keeps it fairly model-centric instead of system-wide, even though some use it more broadly. I would place it between data science and software engineering rather than as a replacement for either: it inherits from data science the empirical, non-specifiable nature of ML, but applies software-engineering discipline, such as automation, testing, and monitoring, to the pipeline around the model. It is not the same as SE4AI in the broader sense used in the course, because SE4AI also covers requirements and non-ML components, while AI Engineering tends to stop at the model and its pipeline.

4.5 Foundation models, AI coding assistants, RAG, and agentic workflows

The clean boundary the book relies on, a trained model with a defined interface that a software engineer can treat as an opaque but stable component, becomes much blurrier once the "model" is an external foundation model accessed through prompts. What is actually built is a pipeline of prompts, retrieval steps, and orchestration logic. It is written as ordinary code and looks like software engineering, but it still behaves like an ML component, because it has no real specification and must be evaluated empirically instead of proven correct.

This pushes software engineers toward work that looks like data science, since they end up evaluating and iterating on prompt behavior empirically. It pushes the people who build agentic systems toward more traditional software engineering skills, because there is no training step left to hide the complexity behind, only an orchestration layer that needs testing and state management like any other system. This is the "blur zone" from the first lecture, and it is relevant for where my own research could go next: robotic systems increasingly chain a foundation model, a perception model, and a planner together, so the interesting attack surface may stop being any single model and become the orchestration layer connecting them.

5 Paper analysis

5.1 LLM-Based Safety Case Generation for Baidu Apollo: Are We There Yet? (Odu, Belle & Wang, CAIN 2025)

5.1.1 Core ideas and SE importance

The paper builds a safety case, that is a structured argument backed by evidence, for the ML-enabled trajectory-prediction component of Baidu Apollo, an open-source autonomous driving system [1]. The authors adapt AMLAS, the standard guidance for assuring machine learning in autonomous systems, and follow a process that starts from hazard analysis: they define system functions, derive malfunctions, specify operational scenarios, and distill safety goals, and then encode the resulting argument in Goal Structuring Notation. On top of the manual process, they test whether GPT-4o can recreate the same safety case automatically. They find that it does so convincingly only when it is given rich context, domain knowledge, and a predicate-based formalization of the argument pattern, and not in a bare zero-shot setting. This matters for AI engineering for two reasons. First, it treats safety assurance as an explicit lifecycle deliverable for ML components, instead of something bolted on once the model works. Second, it gives a concrete and rather sobering data point on where LLM-assisted tooling currently helps and where it still needs human scaffolding.

5.1.2 Relation to my research

My resilience-framework project shares the same underlying impulse: do not simply claim that a cyber-physical system is safe or resilient, but structure and evaluate that claim against a defined set of scenarios. Where their argument is built from operational scenarios and malfunctions expressed in GSN, mine is built from eight attack scenarios evaluated against three formal predicates for detection, tolerance, and recovery. The difference is mainly one of framing and direction: their safety case is retrospective, assuring a system that is already deployed, while my work is adversarial and evaluative, testing whether the resilience claims of a robot really hold under attack in simulation. In a sense, my predicates could be read as one instantiation of the evidence that a security case would need to supply.

5.1.3 Integration into a larger AI-intensive project

Picture a warehouse logistics fleet of autonomous robots doing picking and transport. It combines a perception and trajectory-planning stack, a Whisper-based voice interface for operator commands, and RL-trained manipulation policies, all reporting into a cloud fleet-management dashboard. The ideas of this paper would apply almost directly. Before deploying a new perception or planning model, the safety team could build an AMLAS-style safety case for it, using hazard analysis and GSN to structure the argument, and using LLM assistance to speed up the documentation across many robot variants and software versions. As the paper shows, this would only work if the LLM is given the same kind of structured context and domain knowledge.

My own research would sit under that argument as the evidence layer. The resilience predicates would supply concrete detect, tolerate, and recover evidence nodes for the planning and perception components. The speech-backdoor results, and later the RL-poisoning results once they are complete, would

populate the threat and defeater side of a security case for the voice interface and for the manipulation policies. In other words, they would provide the negative results and the known gaps that a rigorous assurance case is supposed to track instead of hide.

5.1.4 Adaptation of my research

To support the assurance-case idea in the long term, I would want to make my resilience predicates pluggable into a GSN-style argument instead of leaving them in a standalone, theorem-style writeup, structuring each of the eight attack scenarios as an explicit goal-strategy-solution fragment. I would also formalize the scenario descriptions and results in a predicate-based format similar to the one the paper uses, so that a future safety-case tool could ingest them directly instead of requiring someone to transcribe them by hand. Finally, an independent review step, where a second person rates the resilience argument against explicit quality criteria as the reviewers do in the paper, would bring my evaluation closer to a real assurance artifact and not only a simulation result.

5.2 BAFFLE: Hiding Backdoors in Offline Reinforcement Learning Datasets (Gong et al., IEEE S&P 2024)

5.2.1 Core ideas and SE importance

BAFFLE poisons the datasets used to train offline RL agents, that is agents trained purely from a fixed, pre-collected dataset without further interaction with the environment [2]. This is the kind of shared artifact that gets downloaded and reused across many projects. The attack is agent-agnostic: it works without knowing which offline RL algorithm the victim will use, because it injects "misleading experiences" that pair a trigger with a deliberately bad action and an inflated reward. The authors test it on four control tasks, three of which are robotic, and on nine offline RL algorithms covering value-based, policy-based, and actor-critic families. The attack barely affects performance in clean conditions, but it causes a severe drop as soon as the trigger appears, and the backdoor survives a later fine-tuning on clean data. Standard backdoor-detection techniques largely fail to catch it. The paper matters for AI engineering because it reframes what looks like a data-quality or MLOps concern, trusting a shared training dataset, into a genuine software-supply-chain security threat, and because it shows that existing ML defenses are not sufficient.

5.2.2 Relation to my research

This paper overlaps with my reinforcement-learning thread on backdoor poisoning of robotic-arm RL controllers, which, as I said in the introduction, is still in progress. Both attack the training artifact instead of the model architecture, both target robotic control, and both frame the attack as a supply-chain problem rather than as a one-off adversarial input. The main difference is scope. BAFFLE poisons the offline dataset alone and produces an untargeted, action-degrading backdoor, while my taxonomy covers reward, action, and state poisoning during online training with PPO. That is a broader attack surface, but I do not yet have the kind of large-scale detection-evasion evaluation that BAFFLE carries out. The paper also connects to our speech-recognition work: both projects poison a training-time artifact with a hidden trigger, audio fine-tuning data in one case and RL trajectories in the other. This suggests that poisoning at fine-tuning time is a cross-modal ML engineering problem, and not something specific to one model family.

5.2.3 Integration into a larger AI-intensive project

Staying with the same warehouse robot fleet, suppose that the manipulation policies are periodically retrained with an offline RL pipeline, on trajectory datasets contributed by several client sites. This is a realistic fleet-learning setup, where the central team does not fully control or audit the data of every contributor, and it is precisely the threat model BAFFLE targets. The paper would motivate the team to

build a data-provenance and holdout-validation pipeline before ingesting external datasets, and to treat the near-zero effect on clean performance reported by BAFFLE as a warning that ordinary accuracy checks will not catch this attack.

My research would extend that picture in two ways. The RL-poisoning taxonomy would cover the parts of the lifecycle BAFFLE does not, that is attacks introduced during the online fine-tuning performed by the fleet itself, and not only at dataset ingestion. The resilience predicates would then act as the runtime evaluation harness for this component, asking whether the fleet detects anomalous behavior once a trigger fires, whether it can keep operating safely, and whether it can recover, for example by failing over to a known-safe policy.

5.2.4 Adaptation of my research

The clearest gap BAFFLE exposes is that standard backdoor-detection methods fail against RL poisoning. A natural long-term adaptation of my project is therefore to stop measuring only attack success and agent degradation, and to start testing whether detection techniques such as activation clustering or spectral signatures actually catch my reward, action, and state poisoning attacks, producing a result comparable to theirs. I would also test my attacks in an offline setting, on a fixed dataset instead of only with online PPO training, to see whether they transfer the way the dataset-only attack of BAFFLE does. Finally, a persistence check, retraining a poisoned policy on clean data and observing whether the backdoor survives, would let me confirm whether my attacks share the most worrying property of BAFFLE: that a straightforward retraining pass is not enough to clean a compromised system.

6 Evidence, originality, and GenAI-use transparency

6.1 Main sources

- *LLM-Based Safety Case Generation for Baidu Apollo: Are We There Yet?* [1]. Peer-reviewed conference paper (CAIN 2025, co-located with ICSE 2025). Paper 1 of the analysis, read in full.
- *BAFFLE: Hiding Backdoors in Offline Reinforcement Learning Datasets* [2]. Peer-reviewed conference paper (IEEE S&P 2024, a top-tier security venue). Paper 2, read in full.
- Kästner, C., *Machine Learning in Production*, chapters "1. Introduction" and "2. From Models to Systems". Book chapters from the CMU online book, assigned as course reading. Used for Section 4.
- Course lecture slides, the keynote on agentic software engineering, and the two guest lectures by Frattini and Lenberg. Teaching material. Used for Sections 2 and 3.
- My own research: the resilience-framework paper (accepted at ACM CCS CPSIoTSec), the Whisper speech-backdoor work (in progress), the RL poisoning taxonomy (early stage).

6.2 Which sources I trust most

I trust the two conference papers most, because both passed peer review at established venues and both report enough experimental detail to be checked or reproduced. Between them I have slightly more confidence in BAFFLE, since its evaluation covers nine algorithms and four tasks, which makes the conclusion harder to explain away as an artifact of one particular setup. The CMU chapters are not peer-reviewed in the same way, but they are widely used teaching material written by active researchers, so I treat them as a reliable framing of the area rather than as evidence for a specific empirical claim. I treat the lecture slides similarly: useful as pointers and for organizing my thinking, but compressed, so where a slide referred to an original study I relied on the argument rather than on the slide.

I am most careful with my own unpublished work. The resilience paper has been reviewed and accepted, so I am reasonably confident about it, but the speech-backdoor, RL poisoning have not been

externally validated. I have tried to say this clearly wherever I mention them, so that the essay does not present work in progress as if it were finished.

6.3 Use of generative AI

I used a generative AI assistant in a supporting role, not to produce the analysis. I used it to check my English, since it is not my first language, and to point out passages that were repetitive or unclear.

References

[1] O. Odu, A. B. Belle, and S. Wang. LLM-based safety case generation for Baidu Apollo: Are we there yet? In *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, pages 222–233, 2025. doi:10.1109/CAIN66642.2025.00033.

[2] C. Gong, Z. Yang, Y. Bai, J. He, J. Shi, K. Li, A. Sinha, B. Xu, X. Hou, D. Lo, and T. Wang. Baffle: Hiding backdoors in offline reinforcement learning datasets. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 2086–2104, 2024. doi:10.1109/SP54263.2024.00224.